

What Are the Nice Matrices?

What is Easy and What is Hard?

Matrix multiplication is *easy*, $\mathbf{b} = \mathbf{A}\mathbf{x}$, i.e. evaluate $\mathbf{b}$ given $\mathbf{A}$ and $\mathbf{x}$

$$\begin{pmatrix} b_0 \\ b_1 \\ \dots \end{pmatrix} = \begin{pmatrix} a_{00} & a_{01} & \dots \\ a_{10} & a_{11} & \dots \\ \dots & \dots & \dots \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ \dots \end{pmatrix}$$

Matrix solving is *hard*, i.e. given $\mathbf{A}$ and $\mathbf{b}$ find the column $\mathbf{x}$ so that $\mathbf{A}\mathbf{x} = \mathbf{b}$

$$\begin{pmatrix} a_{00} & a_{01} & \dots \\ a_{10} & a_{11} & \dots \\ \dots & \dots & \dots \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ \dots \end{pmatrix} = \begin{pmatrix} b_0 \\ b_1 \\ \dots \end{pmatrix}$$

I regard this as the single most important message that you must get from this course

Unfortunately, matrix solving is what the most valuable engineering maths demands

What do I mean by *hard* ?

What is Easy and What is Hard?

What do I mean by *hard* ?

I don't mean it's hard for us to learn how to do, I mean that compared to multiplication, it is really really difficult to reliably find the answer and/or computationally expensive in "CPU time/memory"

It's actually worse than that!

Given A and b find the column x so that $Ax = b$ - well actually

- It may be that there isn't any x that "solves" the problem
- It may be that there are many different columns x that "solve" the problem - one problem, many possible answers - er what?
- Even if there does exist a single solution x , it maybe virtually impossible to identify it *numerically*

Now you might understand why I propose exploring **"What Are the Nice Matrices?"** first

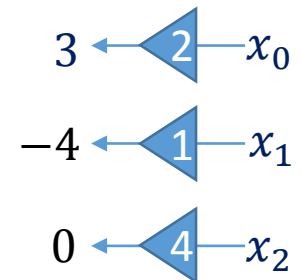
Diagonal Matrices

Square Diagonal matrices only have non-zero elements on the diagonal

Square Diagonal matrix solving is trivial

Example: Given the diagonal matrix problem $\begin{pmatrix} 3 \\ -4 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix}$ solve for the column x

Solution: We might usefully recall the circuit picture of such a diagonal matrix. Surely $x_0 = \frac{3}{2}$, $x_1 = \frac{-4}{1}$, $x_2 = \frac{0}{4}$



We can solve any square diagonal matrix problem as long as no diagonal element is 0

Bidiagonal Matrices

Bidiagonal matrices are square and only have non-zero elements on the diagonal *and* either the diagonal above or below the main diagonal

Example: *The following are lower bidiagonal matrices*

$$\begin{pmatrix} 2 & 0 & 0 \\ 7 & 1 & 0 \\ 0 & 6 & 4 \end{pmatrix} \quad \begin{pmatrix} 2 & 0 \\ 1 & 6 \end{pmatrix} \quad \begin{pmatrix} 2 & 0 & 0 & 0 \\ 7 & 1 & 0 & 0 \\ 0 & 6 & 4 & 0 \\ 0 & 0 & -2 & 7 \end{pmatrix}$$

Example: *The following are upper bidiagonal matrices*

$$\begin{pmatrix} 2 & 7 & 0 \\ 0 & 1 & 5 \\ 0 & 0 & 4 \end{pmatrix} \quad \begin{pmatrix} 2 & 3 \\ 0 & 6 \end{pmatrix} \quad \begin{pmatrix} 2 & 3 & 0 & 0 \\ 0 & 1 & -4 & 0 \\ 0 & 0 & 4 & 1 \\ 0 & 0 & 0 & 7 \end{pmatrix}$$

Bidiagonal matrix solving is fairly trivial

Bidiagonal Matrices

Example: Given the bidiagonal matrix problem $\begin{pmatrix} 5 \\ -4 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 0 \\ 0 & 1 & 8 \\ 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix}$
solve for the column x

Solution: Note how one row always has only one non-zero element

$$\begin{pmatrix} 5 \\ -4 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 0 \\ 0 & 1 & 8 \\ 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix}$$

This row represents the equation $1 = 4x_2$ which means we can say $x_2 = 0.25$

If we now use this we then get

$$\begin{pmatrix} 5 \\ -4 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 0 \\ 0 & 1 & 8 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ 0.25 \end{pmatrix}$$

Now the 2nd last row only involves one unknown, $-4 = 1x_1 + 8 * 0.25$ hence $x_1 = -5$. If we also use this we then get

$$\begin{pmatrix} 5 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ -5 \\ 0.25 \end{pmatrix} \quad \text{hence } 5 = 2x_0 + 3 * (-5) , \text{ hence } x_0 = 10.$$

$$x = \begin{pmatrix} 0.25 \\ -5 \\ 10 \end{pmatrix}$$

Bidiagonal Matrices

Spot the pattern?

Upper bidiagonal matrices

$$\begin{pmatrix} A_{00} & A_{01} & & \\ & A_{11} & A_{12} & \\ & & A_{22} & A_{23} \\ & & & A_{33} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} \quad \text{for } N=3$$

Start with the last row, $x_{N-1} = \frac{b_{N-1}}{A_{N-1,N-1}}$

and work upward $x_i = \frac{b_i - A_{i,i+1} x_{i+1}}{A_{i,i}}$

Lower bidiagonal matrices

$$\begin{pmatrix} A_{00} & & & \\ A_{10} & A_{11} & & \\ & A_{21} & A_{22} & \\ & & A_{32} & A_{33} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} \quad \text{for } N=3$$

Start with the first row, $x_0 = \frac{b_0}{A_{0,0}}$

and work downward $x_i = \frac{b_i - A_{i,i-1} x_{i-1}}{A_{i,i}}$

Triangular Matrices

Spot the generalisation to triangular matrices?

Upper triangular matrices

$$\begin{pmatrix} A_{00} & A_{01} & A_{02} & A_{03} \\ & A_{11} & A_{12} & A_{13} \\ & & A_{22} & A_{23} \\ & & & A_{33} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} \quad \text{for } N=3$$

Start with the last row, $x_{N-1} = \frac{b_{N-1}}{A_{N-1,N-1}}$

and work upward $x_i = \frac{b_i - A_{i,i+1} x_{i+1} - A_{i,i+2} x_{i+2} \dots - A_{i,N-1} x_{N-1}}{A_{i,i}}$

Lower triangular matrices

$$\begin{pmatrix} A_{00} & & & \\ A_{10} & A_{11} & & \\ A_{20} & A_{21} & A_{22} & \\ A_{30} & A_{31} & A_{32} & A_{33} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} \quad \text{for } N=3$$

Start with the first row, $x_0 = \frac{b_0}{A_{0,0}}$

and work downward $x_i = \frac{b_i - A_{i,i-1} x_{i-1} - A_{i,i-2} x_{i-2} \dots - A_{i,0} x_0}{A_{i,i}}$

Triangular Matrices

Example: Solve the triangular matrix problem
$$\begin{pmatrix} 1 & 2 & 3 & -4 \\ & 2 & -2 & 2 \\ & & 3 & 0 \\ & & & 2 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 3 \\ 1 \end{pmatrix}$$

Solution:

From 4th row, $x_3 = 0.5$

From 3rd row, $x_2 = 1$

From 2nd row, $2x_1 - 2(1) + 2(0.5) = 1 \rightarrow x_1 = 1$

From 1st row, $x_0 + 2(1) + 3(1) - 4(0.5) = 0 \rightarrow x_0 = -3$

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} -3 \\ 1 \\ 1 \\ 0.5 \end{pmatrix}$$

What Are the Nice Matrices?

(i.e. for solving $Ax = b$)

Diagonal, bidiagonal, triangular - but that's about it!

Manipulating Matrices into a Nicer Form

Maybe we can change the ordering of the unknowns/equations of a "nasty" matrix to obtain a "nice" matrix?

Example: Given the matrix problem $\begin{pmatrix} 5 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 4 \\ 2 & 3 & 7 \\ 0 & 5 & 8 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix}$ solve for the column x

Solution: We *might* notice that a row swap will leave a triangular matrix

$$\begin{pmatrix} 3 \\ 1 \\ 5 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 7 \\ 0 & 5 & 8 \\ 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} \text{ which is now solvable}$$

Generally, we could propose searching for row and column permutation matrices so that we get an equivalent triangular matrix problem

Unfortunately, the chances of this working are highly unlikely - impossibly so!

Manipulating Matrices into a Nicer Form

We may have failed in the last idea, but

Useful Observation: *Given $Ax = b$ multiplying both sides by another suitable sized matrix such as M does not affect the "engineering" being represented, but it *does* change the form in which we express it.*

By analogy, compare with scaling the simple expression $y=ax$ to $2y=2ax$, i.e. $(2y) = (2a)x$, i.e. $y' = a'x$ - clearly still the same "engineering"

Some thinking: Let's think about the simplest case we can, a 2 by 2 matrix problem to get some inspiration

$$\begin{pmatrix} b_0 \\ b_1 \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} \quad \text{now choose to multiply both sides by the same matrix}$$

$$\begin{pmatrix} M_{00} & M_{01} \\ M_{10} & M_{11} \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \end{pmatrix} = \begin{pmatrix} M_{00} & M_{01} \\ M_{10} & M_{11} \end{pmatrix} \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$$

Remember the elements of M are up to us - can we select them helpfully?

Manipulating Matrices into a Nicer Form

$$\begin{pmatrix} M_{00}b_0 + M_{01}b_1 \\ M_{10}b_0 + M_{11}b_1 \end{pmatrix} = \begin{pmatrix} M_{00}A_{00} + M_{01}A_{10} & M_{00}A_{01} + M_{01}A_{11} \\ M_{10}A_{00} + M_{11}A_{10} & M_{10}A_{01} + M_{11}A_{11} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$$

We are targeting getting a triangular matrix - let's choose lower triangular

$$\begin{pmatrix} M_{00}A_{00} + M_{01}A_{10} & M_{00}A_{01} + M_{01}A_{11} \\ M_{10}A_{00} + M_{11}A_{10} & M_{10}A_{01} + M_{11}A_{11} \end{pmatrix} = \begin{pmatrix} * & 0 \\ * & * \end{pmatrix}$$

All we need to do is select the elements M_{00} and M_{01} so that $M_{00}A_{01} + M_{01}A_{11} = 0$

Actually, there are many possible choices - indeed we have given ourselves too much flexibility - but the key point is yes, for this 2 by 2 problem we can always find a suitable matrix M such that $A' = MA$ is lower triangular and hence solvable.

For reasons we'll explore later, we will select $M = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ for some angle θ

Manipulating Matrices into a Nicer Form

Example: Given the matrix problem $\begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 & 7 \\ 5 & 8 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$ solve for the column x

Solution: Manipulate to lower triangular form by multiplying through

$$\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} 3 & 7 \\ 5 & 8 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$$

$$\begin{pmatrix} 3 \cos \theta - 1 \sin \theta \\ 3 \sin \theta + 1 \cos \theta \end{pmatrix} = \begin{pmatrix} 3 \cos \theta - 5 \sin \theta & 7 \cos \theta - 8 \sin \theta \\ 3 \sin \theta + 5 \cos \theta & 7 \sin \theta + 8 \cos \theta \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$$

I choose to make $7 \cos \theta - 8 \sin \theta = 0$, i.e. by selecting $\theta = \tan^{-1} \left(\frac{7}{8} \right)$

Using this value now gives

$$\begin{pmatrix} 1.599225476 \\ 2.728090518 \end{pmatrix} = \begin{pmatrix} -1.034792955 & 0 \\ 5.738397297 & 10.63014581 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}$$

$$x_0 = \frac{1.599225476}{-1.034792955} = -1.545454546$$

$$x_1 = \frac{2.728090518 - 5.73839729 * (-1.545454546)}{10.63014581} = 1.090909091$$

$$x = \begin{pmatrix} -1.55 \\ 1.091 \end{pmatrix}$$

Manipulating Matrices into a Nicer Form

OK for 2 by 2 matrices, but what about 3 by 3 and beyond?

Lateral thinking:

1) Note how the helpful matrix we used, $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ is a rotation matrix (see slide ??)

2) On slide ?? we constructed "higher dimensional rotations" by cascading simpler rotations

So maybe a general rotation is a cascade of basic simple rotations?

Manipulating Matrices into a Nicer Form

Example: Evaluate the matrix product

$$\begin{pmatrix} 1 & & & & \\ & \cos \theta & & -\sin \theta & \\ & & 1 & & \\ & \sin \theta & & \cos \theta & \\ & & & & 1 \end{pmatrix} \begin{pmatrix} A_{00} & A_{01} & A_{02} & A_{03} & A_{04} \\ A_{10} & A_{11} & A_{12} & A_{13} & A_{14} \\ A_{20} & A_{21} & A_{22} & A_{23} & A_{24} \\ A_{30} & A_{31} & A_{32} & A_{33} & A_{34} \\ A_{40} & A_{41} & A_{42} & A_{43} & A_{44} \end{pmatrix}$$

Solution:

$$\begin{pmatrix} A_{00} & A_{01} & A_{02} & A_{03} & A_{04} \\ \cos \theta A_{10} - \sin \theta A_{30} & \cos \theta A_{11} - \sin \theta A_{31} & \cos \theta A_{12} - \sin \theta A_{32} & \cos \theta A_{13} - \sin \theta A_{33} & \cos \theta A_{14} - \sin \theta A_{34} \\ A_{20} & A_{21} & A_{22} & A_{23} & A_{24} \\ \sin \theta A_{10} + \cos \theta A_{30} & \sin \theta A_{11} + \cos \theta A_{31} & \sin \theta A_{12} + \cos \theta A_{32} & \sin \theta A_{13} + \cos \theta A_{33} & \sin \theta A_{14} + \cos \theta A_{34} \\ A_{40} & A_{41} & A_{42} & A_{43} & A_{44} \end{pmatrix}$$

Key Observation: Only two rows are changed and we could select θ to "zero" any one element we wish in either of those two rows

So if the "cos, sin" terms appear in **columns** j and j' of the **first** matrix, then we are effectively changing the values in **rows** j and j' of the **second** matrix

Manipulating Matrices into a Nicer Form

Example: Choose a suitable (rotation) matrix, R , such that

$$A' = R \begin{pmatrix} A_{00} & A_{01} & A_{02} & A_{03} & A_{04} \\ A_{10} & A_{11} & A_{12} & A_{13} & A_{14} \\ A_{20} & A_{21} & A_{22} & A_{23} & A_{24} \\ A_{30} & A_{31} & A_{32} & A_{33} & A_{34} \\ A_{40} & A_{41} & A_{42} & A_{43} & A_{44} \end{pmatrix} \text{ has a zero for element } A'_{03}$$

Solution: We must have the "cos, sin" in the 0th column of R . The choice of the second column is arbitrary, let's say the 5th one. Moreover, the choice for the "lower row" is also free to choose, but for those other reasons, we will make the rows the same as the columns

$$R = \begin{pmatrix} \cos \theta & & & -\sin \theta \\ & 1 & & \\ & & 1 & \\ & & & 1 \\ \sin \theta & & & \cos \theta \end{pmatrix}$$

$$A'_{03} = \cos \theta A_{03} - \sin \theta A_{43}, \text{ so choose } \theta = \tan^{-1} \left(\frac{A_{03}}{A_{43}} \right)$$

Manipulating Matrices into a Nicer Form

Key Observations:

We seem to have a lot of freedom here - sounds good cos we intend to cascade such operations

Some (most) elements of $\mathbf{A}$ are unaffected by $\mathbf{R}$

Strategy: *Multiply by a sequence of such rotations each of which targets zeroing a particular element in $\mathbf{A}$*

Be Careful: *Once an element is zeroed, mustn't change it by the later rotations - that's what we will use those remaining "free choices" for*

Manipulating Matrices into a Nicer Form

Note the pattern: A rotation matrix with the "cos, sin" elements in rows/columns i, k , changes all the elements in rows i and k of the matrix it is multiplying

$$\begin{array}{c} \text{Column } i \\ \text{Column } k \end{array} \begin{array}{c} \text{Row } i \\ \text{Row } k \end{array} \begin{pmatrix} 1 & & & & \\ & \cos \theta & & -\sin \theta & \\ & \sin \theta & 1 & \cos \theta & \\ & & & & 1 \end{pmatrix}$$

The particular rotation matrix form we are using is called a **Given's Rotation**

$$\begin{array}{c} \text{Row } i \\ \text{Row } k \end{array} \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & \cdot \\ * & * & * & * & * \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ * & * & * & * & * \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix}$$

• Unaffected
 * Changed

θ is chosen to make one of the * elements zero

Manipulating Matrices into a Nicer Form

Example: Find a sequence of Given's rotations that reduces a 3 by 3 matrix to lower triangular form

$$A = \begin{pmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{pmatrix}$$

Solution:

1) Target zeroing the A_{20} element

$$\begin{pmatrix} 1 & & \\ \cos \theta_0 & -\sin \theta_0 & \\ \sin \theta_0 & \cos \theta_0 & \end{pmatrix} \begin{pmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} & A_{02} \\ * & * & * \\ 0 & * & * \end{pmatrix}$$

2) Target zeroing the A_{10} element

$$\begin{pmatrix} \cos \theta_1 & -\sin \theta_1 & \\ \sin \theta_1 & \cos \theta_1 & \\ & & 1 \end{pmatrix} \begin{pmatrix} A_{00} & A_{01} & A_{02} \\ * & * & * \\ 0 & * & * \end{pmatrix} = \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & * & * \end{pmatrix}$$

3) Target zeroing the A_{21} element

$$\begin{pmatrix} 1 & & \\ \cos \theta_2 & -\sin \theta_2 & \\ \sin \theta_2 & \cos \theta_2 & \end{pmatrix} \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & * & * \end{pmatrix} = \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \end{pmatrix}$$

Manipulating Matrices into a Nicer Form

Can we spot a "computer programmable pattern"

Example: Find a sequence of Given's rotations that reduce a 5 by 5 matrix to lower triangular form

Solution: We'll have to compress the notation and just show the key issues

$$A = \begin{pmatrix} A_{00} & A_{01} & A_{02} & A_{03} & A_{04} \\ A_{10} & A_{11} & A_{12} & A_{13} & A_{14} \\ A_{20} & A_{21} & A_{22} & A_{23} & A_{24} \\ A_{30} & A_{31} & A_{32} & A_{33} & A_{34} \\ A_{40} & A_{41} & A_{42} & A_{43} & A_{44} \end{pmatrix}$$

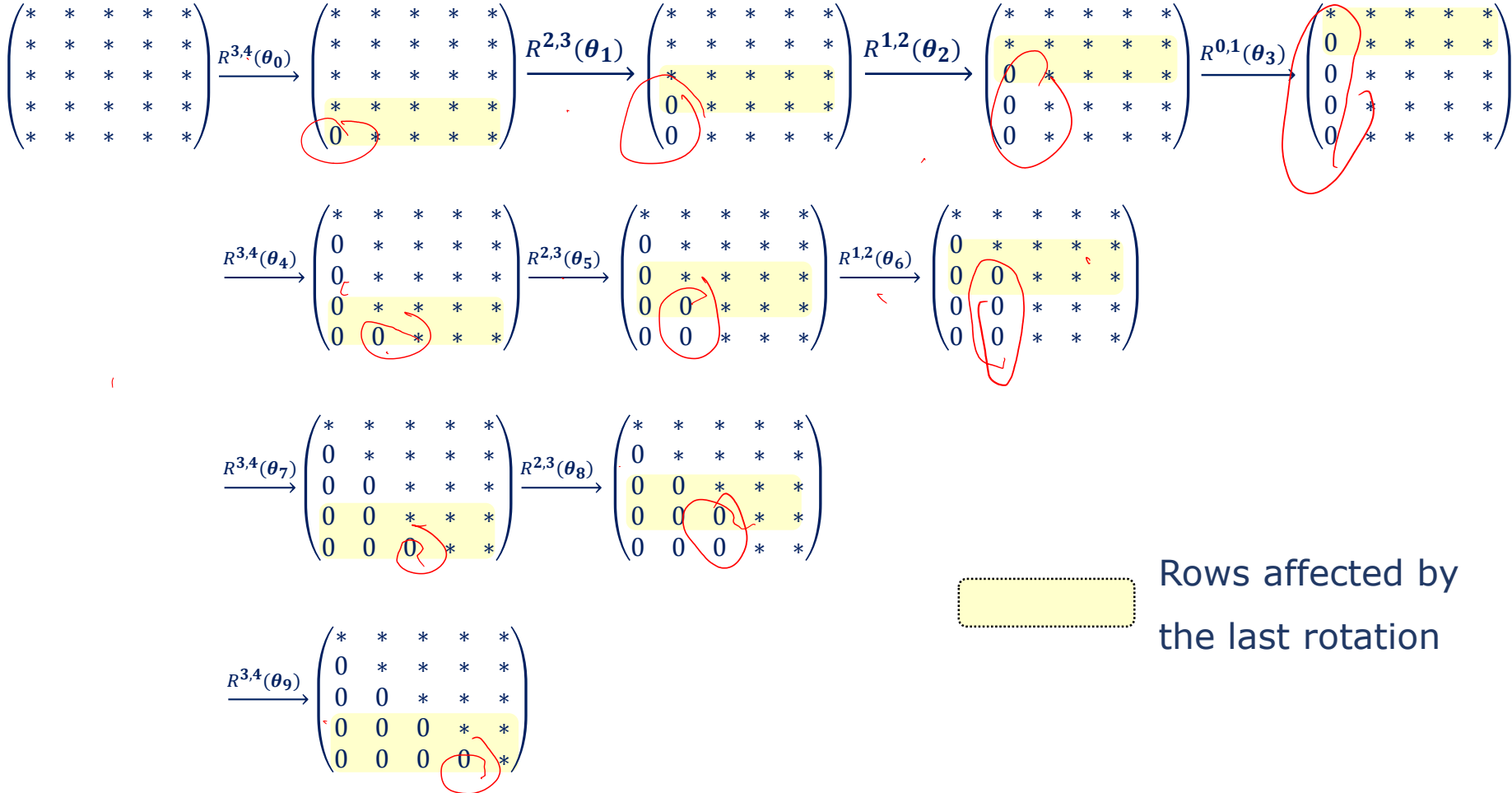
The recommended order in which to target the elements

Denote a Given's Rotation that has the "cos, sin" terms in rows and columns i,k as

$$R^{i,k}(\theta)$$

$$\begin{matrix} & \text{Column } i & & \text{Column } k \\ \text{Row } i & \begin{pmatrix} 1 & \cos \theta & -\sin \theta \\ \sin \theta & 1 & \cos \theta \end{pmatrix} & & \\ \text{Row } k & & & \end{matrix}$$

Manipulating Matrices into a Nicer Form



Looks good for developing a programmable algorithm?

Manipulating Matrices into a Nicer Form

Summary: Haven't we just identified a method of solving $Ax = b$?

We have shown that lower (upper) tridiagonal matrices can be solved

$$\begin{pmatrix} \text{blue triangle} & \text{red X's} \end{pmatrix} \begin{pmatrix} * \\ * \\ * \end{pmatrix} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}$$

We sought to manipulate $Ax = b$ into $A'x = b'$ where A' is a nicer matrix than A

➤ We can use a Given's Rotations to force the appearance of an 0 in a matrix

$$\begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & 1 & & \\ & & & \cos \theta & -\sin \theta \\ & & & \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{pmatrix} = \begin{pmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ 0 & * & * & * & * \end{pmatrix}$$

➤ Sequential use of Given's rotations can reduce any square A to lower (upper) triangular form

➤ Standard triangular matrix solving

Manipulating Matrices into a Nicer Form

Summary: Haven't we just identified a method of solving $Ax = b$?

$$Ax = b$$

$$R_0Ax = R_0b$$

$$R_1R_0Ax = R_1R_0b$$

$$R_2R_1R_0Ax = R_2R_1R_0b$$

$$R_N \dots R_1R_0Ax = R_N \dots R_2R_1R_0b = [R_N \dots R_1R_0A]x = [R_N \dots R_2R_1R_0b] = A'x = b'$$

Handwritten in red: A triangular.

Success - but now those recurring engineering questions

What's the Cost?

What's the Risk?

Assessing the Cost

The massive questions are;

What's the Cost? - Matrix operations take computer time to do!

1) If we have $A^{(N \times N)} x^{(N \times 1)} = b^{(N \times 1)}$ and "solve for x" how many cpu seconds does it require, in particular how does this time *scale* with N

What's the Risk? - will it always work and will the "answer be reliable"?

2) If we are doing a zillion multiplies and adds on a CPU with finite precision are we confident that the true result doesn't get swamped by *rounding noise* ?

3) Actually, we did comment earlier that there may not exist a unique solution for x - we must explore that

First let us start counting the computational cost

Assessing the Cost

The recurrent building block of all matrix programs that consumes CPU seconds is the "scalar multiply and add", i.e. $p = q + r * s$. Hence this is the unit measure of effort

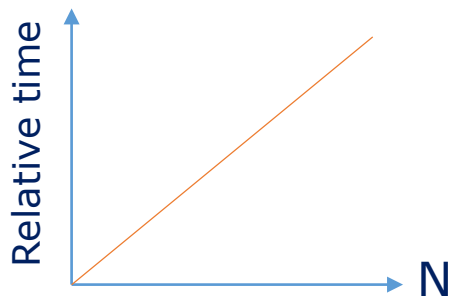
Example: How does evaluating the norm of a column vector scale with size?

$$\|V\|_2^2 = V^T V$$

Solution: Recall that

$$\|V\|_2^2 = V^T V = (v_0 \quad v_1 \quad \dots \quad v_{N-1}) \begin{pmatrix} v_0 \\ v_1 \\ \dots \\ v_{N-1} \end{pmatrix} = \sum_{j=0}^{N-1} v_j \quad v_j$$

evaluating the sum requires N "multiplies and adds". As N varies we might plot



Linear - doubling the matrix size doubles the effort

Assessing the Cost

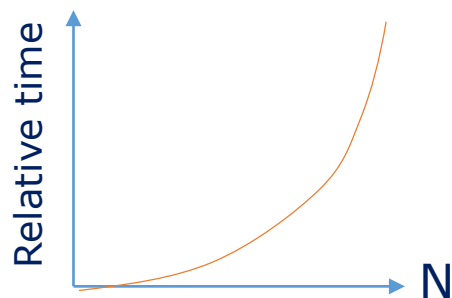
Example: How does evaluating the matrix product of two square matrices scale?

$$P^{(N \times N)} = U^{(N \times N)} V^{(N \times N)}$$

Solution: Recall the operation to get *each* element of the result

$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} \begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} = \begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix}$$

We have to evaluate N rows and N columns of the product - N^2 elements, each of which requires an N term sum to get - overall N^3 "multiplies and adds".



Cubic - doubling the matrix size *8 the effort

*This is extremely unpleasant. If
an $N=500$ problem needs 1 second to evaluate,
an $N= 5000$ problem needs 1000s and
an $N= 50000$ problem, 1000000s (11.6 days!)*

What's the Cost?

We use the notation $O(N^p)$ - O for Order, it's not the actual number of seconds that matters it's the scaling (i.e. change of effort with change in size, N)

$$(* \quad * \quad *) \begin{pmatrix} * \\ * \\ * \end{pmatrix} = (*) \quad O(N)$$

$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} \begin{pmatrix} * \\ * \\ * \end{pmatrix} = \begin{pmatrix} * \\ * \\ * \end{pmatrix} \quad O(N^2)$$

$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} \begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} = \begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} O(N^3) \quad \text{X 10}$$

What's the Cost?

In such a short course we won't do more than:

- 1) Note this is a critical "measure of success" for any numerical algorithm
- 2) Note that $O(N)$ is the best one can usually dream of and $O(N^3)$ is dire. For reference, note that many successful algorithms are $O(N \log N)$ (FFT)
- 3) In a later session we will ask ourselves about the situation when "most" of the matrix elements are 0, which is a very practical case that we can specialise

$$\begin{pmatrix} * & * \\ * & * \\ * & * \end{pmatrix} \begin{pmatrix} * & * \\ * & * \\ * & * \end{pmatrix}$$

we can make this much better than $O(N^3)$

What Are the Nice Matrices?

Are there any other general properties that a matrix might possess that we might regard as "nice"?

"Self Inverting" Matrices

We commented a while back that the "concept" of an inverse matrix can be practically useless for large-scale numerical calculations. However, it does have its value; A reminder

For a square matrix, A , there may exist another square matrix, denoted by A^{-1} and called the inverse matrix of A , such that $AA^{-1} = A^{-1}A = I$

In principle, the inverse can be used to solve the $Ax = b$ problem

$$x = A^{-1}b$$

$$Ax = b, \quad A^T Ax = A^T b, \quad x = A^{-1}b$$

"Text book" results exist to construct simple inverse matrices, e.g. 2 by 2 , 3 by 3

Example: Given a 2 by 2 matrix $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ it is straightforward to demonstrate

that $A^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$

Large scale matrix solving is virtually never done by constructing an explicit inverse - very inefficient and numerically unreliable

"Self Inverting" Matrices

We have met a type of matrix that does have a very nice inverse - virtually itself

Any matrix, Q whose transpose, Q^T is the inverse of the matrix is called **orthogonal**; hence $Q^T Q = Q Q^T = I$

The case we met before are *permutation matrices*

Example: If we knew that we were dealing with an orthogonal matrix in $Qx = b$ then solving for x is trivial,

$$Q^T Qx = Q^T b, \text{ i.e. } x = Q^T b$$

$$Qx = b$$

$$Q^T Qx = Q^T b, \quad x = Q^T b$$

Orthogonal matrices are numerically wonderful when it comes to dealing with rounding error in finite precision computing - we will explore later.

Does that mean that all our work on using Given's Rotations to solve general $Ax = b$ problems needs to be abandoned in favour of something using orthogonal matrices?

"Self Inverting" Matrices

Example: Show that Given's rotation matrices *are orthogonal*

Solution: Recall

$$(R_1 R_0)^T (R_1 R_0) = I$$

Row i
Row k

$$R = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Column i Column k

Note that

Row i
Row k

$$R^T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \cos \theta & 1 & 0 & 0 \\ -\sin \theta & 0 & 1 & 0 \\ \sin \theta & 0 & 0 & 1 \end{pmatrix}$$

Column i Column k

$$R^T R = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \cos \theta & 1 & 0 & 0 \\ -\sin \theta & 0 & 1 & 0 \\ \sin \theta & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ \cos \theta & 1 & 0 & 0 \\ \sin \theta & 1 & 0 & 0 \\ \cos \theta & 0 & 1 & 0 \end{pmatrix} =$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ \cos^2 \theta + \sin^2 \theta & 1 & 0 & 0 \\ -\sin \theta \cos \theta + \cos \theta \sin \theta & 0 & 1 & 0 \\ \cos^2 \theta + \sin^2 \theta & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

Let's show that this is helpful!

"Self Inverting" Matrices

Example: Exploit the *orthogonality* of Given's rotations in solving $Ax = b$?

Solution: Recall we multiplied through by a sequence of Given's rotations to get a lower triangular A'

$$R_N \dots R_1 R_0 A x = R_N \dots R_2 R_1 R_0 b = [R_N \dots R_1 R_0 A] x = [R_N \dots R_2 R_1 R_0 b] = A' x = b'$$

We don't actually explicitly exploit the orthogonality, it's just that its existence improves the numerical behaviour

What Are the Nice Matrices?

Are there any other general properties that a matrix might possess that we might regard as "nice"?

Orthogonal matrices are really helpful

We have shown that Given's rotations are orthogonal

We might wonder whether the Given's rotation matrices are the *only* way of reducing a matrix to lower or upper triangular form, maybe it's just their orthogonality property that matters?

The QR Decomposition

Time for one of those "it can be shown moments"

It can be shown that: Any matrix can be *decomposed* into the product, $A = QR$, of an orthogonal matrix Q and an upper triangular matrix R , where $Q^T Q = Q Q^T = I$

$$\begin{pmatrix} A_{00} & A_{01} & A_{02} & A_{03} & A_{04} \\ A_{10} & A_{11} & A_{12} & A_{13} & A_{14} \\ A_{20} & A_{21} & A_{22} & A_{23} & A_{24} \\ A_{30} & A_{31} & A_{32} & A_{33} & A_{34} \\ A_{40} & A_{41} & A_{42} & A_{43} & A_{44} \end{pmatrix} = Q \begin{pmatrix} * & * & * & * & * \\ & * & * & * & * \\ & & * & * & * \\ & & & * & * \\ & & & & * \end{pmatrix}$$

$$Ax = b$$

$$QRx = b$$

$$Q^T Q R x = Q^T b$$

$$Rx = Q^T b$$

Doesn't this make solving $Ax = b$ trivial?

Example: Use the QR decomposition to solve $Ax = b$

Solution: $A = QR$; $Q^T Ax = Q^T b$, $Q^T QRx = Q^T b$, $Rx = Q^T b$

$Rx = Q^T b$ is now easily solvable for x as R is triangular

Knowing something exists is not the same as finding it!

The QR decomposition: It can be shown that any matrix can be *decomposed* into the product, $A = QR$...

The theorem says that there exist ***Q*** and ***R*** such that - but is quiet on how to reliably obtain them!

Well we *do have Given's rotations* as a special case don't we?

The QR Decomposition

Example: Show that Given's rotations are a way to get a QR decomposition

Solution: Recall we multiplied through by a sequence of Given's rotations to get a lower triangular A'

$$R_N \dots R_1 R_0 A x = R_N \dots R_2 R_1 R_0 b = [R_N \dots R_1 R_0 A] x = [R_N \dots R_2 R_1 R_0 b] = A' x = b'$$

Unfortunately, R for rotation clashes with the R in the QR theorem so, given that we have shown that each Given's rotation is orthogonal, let's re-annotate this as

$$q_N \dots q_1 q_0 A x = [q_N \dots q_1 q_0 A] x = q_N \dots q_1 q_0 b$$

where we know that $[q_N \dots q_1 q_0 A]$ is a lower triangular matrix, which we might now denote by L

$$[q_N \dots q_1 q_0 A] = L$$

The QR Decomposition

$$[q_N \dots q_1 q_0 A] = L$$

Use the orthogonality of the Given's rotations

$$[q_{N-1} \dots q_1 q_0 A] = q_N^T L$$

$$[q_{N-2} \dots q_1 q_0 A] = q_{N-1}^T q_N^T L$$

...

$$A = q_0^T \dots q_{N-1}^T q_N^T L = (q_{N-1} \dots q_1 q_0)^T L = Q^T L$$

$$A^T = q_0^T \dots q_{N-1}^T q_N^T L = (q_{N-1} \dots q_1 q_0)^T L = RQ$$

Well almost?

$q_h \rightarrow$ given rotation

$R \rightarrow$ upper/lower triangular matrix

$$Ax = b$$

$$q_1 q_0 = q_h$$

$$q_1 q_0 A x = q_1 q_0 b$$

$$q_h A x = q_h b$$

$$R x = q_h b$$

Knowing something exists is not the same as finding it!

Are there other (better, whatever that means) ways of obtaining a QR decomposition?

Why do we care?

COST

RELIABILITY

Householder Reflections

If orthogonal matrices are so useful, maybe we need to identify a few more than just *Permutations* and *Given's Rotations*

Example: Show that if we select any column vector $\mathbf{v}$ and use it to construct a square matrix $\mathbf{H} = \mathbf{I} - \frac{2\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}}$ that $\mathbf{H}$ is orthogonal, i.e. $\mathbf{H}^T\mathbf{H} = \mathbf{1}$

Solution: Let's get a picture of the "shape" of $\mathbf{H}$

First note that $\mathbf{v}^T\mathbf{v} = (v_0 \quad \dots \quad v_{N-1}) \begin{pmatrix} v_0 \\ \dots \\ v_{N-1} \end{pmatrix}$ and so evaluates to a number - hence we can divide by it

Second note that $\mathbf{v}\mathbf{v}^T = \begin{pmatrix} v_0 \\ \dots \\ v_{N-1} \end{pmatrix} (v_0 \quad \dots \quad v_{N-1}) = \begin{pmatrix} v_0v_0 & \dots & v_0v_{N-1} \\ \dots & \dots & \dots \\ v_{N-1}v_0 & \dots & v_{N-1}v_{N-1} \end{pmatrix}$

$$\mathbf{H}^T\mathbf{H} = \left(\mathbf{I} - \frac{2\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}} \right)^T \left(\mathbf{I} - \frac{2\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}} \right) = ?$$

Householder Reflections

$$H^T H = \left(I - \frac{2vv^T}{v^T v} \right)^T \left(I - \frac{2vv^T}{v^T v} \right) = \left(I^T - \frac{2(vv^T)^T}{v^T v} \right) \left(I - \frac{2vv^T}{v^T v} \right)$$

noting that $(vv^T)^T = (v^T)^T (v)^T = vv^T$ and $I^T = I$

$$H^T H = \left(I - \frac{2vv^T}{v^T v} \right) \left(I - \frac{2vv^T}{v^T v} \right)$$

$$H^T H = II - I \frac{2vv^T}{v^T v} - \frac{2vv^T}{v^T v} I + \frac{4vv^T vv^T}{(v^T v)(v^T v)} = I - \frac{4vv^T}{v^T v} + \frac{4v(v^T v)v^T}{(v^T v)(v^T v)} = I$$

We have shown that matrix **H**, known as a **Householder Reflection Matrix**, is orthogonal

Can this help identify a QR decomposition ?

Householder Reflections

Illustration: Given a matrix $\mathbf{A}$
 what happens if we construct a Householder
 reflection from the elements highlighted and
 pre-multiply $\mathbf{A}$ by $\mathbf{H}$

$$\mathbf{A} = \begin{pmatrix} A_{00} & A_{01} & A_{02} & A_{03} & A_{04} \\ A_{10} & A_{11} & A_{12} & A_{13} & A_{14} \\ A_{20} & A_{21} & A_{22} & A_{23} & A_{24} \\ A_{30} & A_{31} & A_{32} & A_{33} & A_{34} \\ A_{40} & A_{41} & A_{42} & A_{43} & A_{44} \end{pmatrix}$$

We won't prove this - try it yourself maybe, rather just quote the result

Choose $\mathbf{v} = \begin{pmatrix} \alpha \\ A_{10} \\ A_{20} \\ A_{30} \\ A_{40} \end{pmatrix}$ where $\alpha^2 = \sum_{i=0}^4 A_{i0}^2$ then

$$\mathbf{H}^T \mathbf{A} = \begin{pmatrix} * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & * & * & * & * \\ 0 & * & * & * & * \\ 0 & * & * & * & * \end{pmatrix}$$

Zeros most of a column in one go - compare to using multiple Givens rotations

Householder Reflections

Illustration: Can we repeat this to obtain a triangular matrix ? *From above,*

$$H_0^T A = \begin{pmatrix} A'_{00} & A'_{01} & A'_{02} & A'_{03} & A'_{04} \\ 0 & A'_{11} & A'_{12} & A'_{13} & A'_{14} \\ 0 & A'_{21} & A'_{22} & A'_{23} & A'_{24} \\ 0 & A'_{31} & A'_{32} & A'_{33} & A'_{34} \\ 0 & A'_{41} & A'_{42} & A'_{43} & A'_{44} \end{pmatrix}$$

Now choose $v = \begin{pmatrix} 0 \\ \alpha_1 \\ A'_{21} \\ A'_{31} \\ A'_{41} \end{pmatrix}$ where $\alpha_1^2 = \sum_{i=1}^4 A_{i1}^2$

$$H_1^T H_0^T A = \begin{pmatrix} A''_{00} & A''_{01} & A''_{02} & A''_{03} & A''_{04} \\ 0 & A''_{11} & A''_{12} & A''_{13} & A''_{14} \\ 0 & 0 & A''_{22} & A''_{23} & A''_{24} \\ 0 & 0 & A''_{32} & A''_{33} & A''_{34} \\ 0 & 0 & A''_{42} & A''_{43} & A''_{44} \end{pmatrix}$$

Now choose $v = \begin{pmatrix} 0 \\ 0 \\ \alpha_2 \\ A''_{32} \\ A''_{42} \end{pmatrix}$ where $\alpha_2^2 = \sum_{i=2}^4 A_{i2}^2$

$$H_2^T H_1^T H_0^T A = \begin{pmatrix} A'''_{00} & A'''_{01} & A'''_{02} & A'''_{03} & A'''_{04} \\ 0 & A'''_{11} & A'''_{12} & A'''_{13} & A'''_{14} \\ 0 & 0 & A'''_{22} & A'''_{23} & A'''_{24} \\ 0 & 0 & 0 & A'''_{33} & A'''_{34} \\ 0 & 0 & 0 & A'''_{43} & A'''_{44} \end{pmatrix}$$

Now choose $v = \begin{pmatrix} 0 \\ 0 \\ 0 \\ \alpha_3 \\ A'''_{43} \end{pmatrix}$ where $\alpha_3^2 = \sum_{i=3}^4 A_{i3}^2$

$$H_2^T H_1^T H_0^T A = \begin{pmatrix} A''''_{00} & A''''_{01} & A''''_{02} & A''''_{03} & A''''_{04} \\ 0 & A''''_{11} & A''''_{12} & A''''_{13} & A''''_{14} \\ 0 & 0 & A''''_{22} & A''''_{23} & A''''_{24} \\ 0 & 0 & 0 & A''''_{33} & A''''_{34} \\ 0 & 0 & 0 & 0 & A''''_{44} \end{pmatrix}$$

Given the orthogonality of the Householder matrices, we have
 $Q^T A = R$, i.e. $A = QR$

Knowing something exists is not the same as finding it!

Householder Reflections are a "column at a time" method for identifying QR decompositions

They are very efficient to implement and thus are widely used (preferred!)

Any other options? (options are good - horses for courses!)

The LU Decomposition

Our successful strategy for matrix solving is **matrix decomposition** - think *factorisation*, of a general matrix into a product of 2 nicer matrices. We don't *have* to use orthogonal matrices as part of this process so hence let's also note that;

It can be shown that square matrices can be decomposed as a product of a lower triangular matrix, L , and an upper triangular matrix, U , i.e.

$$A^{(N \times N)} = L^{(N \times N)} U^{(N \times N)}$$

In this brief course we prefer not to spend time discussing and memorising the standard routines for evaluating the elements of the LU decomposition.

However, we do comment that one must be rather careful of numerical issues as usual

The LU Decomposition

Example: Show how the LU decomposition can be used to solve, $Ax = b$

Solution:

$$Ax = \begin{pmatrix} L_{00} & 0 & 0 & 0 & 0 \\ L_{10} & L_{11} & 0 & 0 & 0 \\ L_{20} & L_{21} & L_{22} & 0 & 0 \\ L_{30} & L_{31} & L_{32} & L_{33} & 0 \\ L_{40} & L_{41} & L_{42} & L_{43} & L_{44} \end{pmatrix} \begin{pmatrix} U_{00} & U_{01} & U_{02} & U_{03} & U_{04} \\ 0 & U_{11} & U_{12} & U_{13} & U_{14} \\ 0 & 0 & U_{22} & U_{23} & U_{24} \\ 0 & 0 & 0 & U_{33} & U_{34} \\ 0 & 0 & 0 & 0 & U_{44} \end{pmatrix} x = b$$

Define a new column, y , as

$$y = \begin{pmatrix} U_{00} & U_{01} & U_{02} & U_{03} & U_{04} \\ 0 & U_{11} & U_{12} & U_{13} & U_{14} \\ 0 & 0 & U_{22} & U_{23} & U_{24} \\ 0 & 0 & 0 & U_{33} & U_{34} \\ 0 & 0 & 0 & 0 & U_{44} \end{pmatrix} x \quad \text{so that} \quad \begin{pmatrix} L_{00} & 0 & 0 & 0 & 0 \\ L_{10} & L_{11} & 0 & 0 & 0 \\ L_{20} & L_{21} & L_{22} & 0 & 0 \\ L_{30} & L_{31} & L_{32} & L_{33} & 0 \\ L_{40} & L_{41} & L_{42} & L_{43} & L_{44} \end{pmatrix} y = b$$

We can easily(?) solve $Ly = b$ for y given that L is triangular and b is known

Then we can easily(?) solve $Ux = y$ given that U is triangular and y is known

Knowing something exists is not the same as finding it!

QR and related techniques

Givens Rotations ①

Householder Reflections ②

LU and related techniques

We don't have the time to compare the merits of these options, but all are used in reality - unlike that theoretical inverse the text books love so much!

Didn't we say that $Ax=b$ may be unsolveable ?

Given A and b find the column x so that $Ax = b$ - well actually

- It may be that there isn't any x that "solves" the problem
- It may be that there are many different columns x that "solve" the problem - one problem, many possible answers - er what?
- Even if there does exist a single solution x , it maybe virtually impossible to identify it *numerically*

Solvability

Actually we do actually have a good idea of this already

Example: If we are setting up simultaneous equations with N unknowns, how many equations do we need in order to be able to get a solution?

Solution: Don't we know already that we need N equations to solve for N unknowns. In matrix notation, this implies a square matrix

For example. *2 equations in 3 unknowns - can't solve* $\begin{pmatrix} 11 \\ 14 \end{pmatrix} = \begin{pmatrix} 3 & 9 & -1 \\ 4 & 8 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$

In what way do we mean "can't solve"? $\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$ that works so it is a *solution*?

Try $\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} ? \\ ? \\ ? \end{pmatrix}$ that also works so is it not also a *solution*?

The problem is **underspecified** - that means that there are many possible solutions so more correctly, we should say that we can't solve the problem **uniquely**

Solvability

So we add another equation to the set to get
$$\begin{pmatrix} 11 \\ 14 \\ 28 \end{pmatrix} = \begin{pmatrix} 3 & 9 & -1 \\ 4 & 8 & 2 \\ 8 & 16 & 4 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

OK? No, the equation added in the 3rd row is not *independent* of the 2 existing equations - Essentially writing down the same thing twice" is a con!

Even if we have a square matrix, we cannot be sure that we have "enough" ***independent*** equations - how to know/check ?

Finally what if we have more equations than unknowns

For example: 4 equations in 3 unknowns - can't solve

$$\begin{pmatrix} 11 \\ 14 \end{pmatrix} = \begin{pmatrix} 3 & 9 & -1 \\ 4 & 8 & 2 \\ 1 & 2 & 2 \\ 6 & 0 & -9 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

The problem is **overspecified** - that means that there is no solution ***at all***

Didn't we say that $Ax=b$ may be unsolveable ?

Negative thinking

It appears that only square matrix problems, $Ax=b$, can be usefully solved, so forget the rest?

Also we have to be careful that a square matrix actually has *independent* rows, otherwise "it's that write the same row twice con" and hence unsolvable

Positive Thinking

Maybe we just haven't realised yet what non-square matrix problems, $Ax=b$, mean to an engineer and so should not just ignore them

Can we meaningfully solve non-square matrix problems?

QR Decomposition - non square cases

We never actually said that triangular matrices have to be square

same

$$\begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \end{pmatrix}$$

$$\begin{pmatrix} * & 0 & 0 & 0 \\ * & * & 0 & 0 \\ * & * & * & 0 \end{pmatrix}$$

both of these count as triangular

and really usefully, the QR decomposition also applies to non-square matrices

Example: Show the patterns of the QR decomposition for non square matrices

Solution:

$$\begin{pmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{pmatrix}^{M \times N} = \begin{pmatrix} Q_{00} & \dots & Q_{M0} \\ \dots & \dots & \dots \\ Q_{M0} & \dots & Q_{M-1,M-1} \end{pmatrix}^{M \times M} \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \end{pmatrix}^{M \times N}$$

$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix}^{M \times N} = \begin{pmatrix} Q_{00} & \dots & Q_{M0} \\ \dots & \dots & \dots \\ Q_{M0} & \dots & Q_{M-1,M-1} \end{pmatrix}^{M \times M} \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \\ 0 & 0 & 0 \end{pmatrix}^{M \times N}$$

Note that the **Q** matrices are always square

QR Decomposition - non square cases

Example: Apply the QR decomposition to the problem of solving the overspecified problem $A^{M \times N} x = b$ ($M > N$) and try to give a picture to interpret the outcome

Solution:

$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \\ * & * & * \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{N \times 1} = Q^{M \times M} \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \\ 0 & 0 & 0 \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{N \times 1} = \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{M \times 1}$$

A x Q R x b

As usual we can use the orthogonality of Q

$$\begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \\ 0 & 0 & 0 \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{N \times 1} = Q^T \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{M \times 1} = \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{M \times 1}$$

b b'

The right hand side is trivial to evaluate and completely known

QR Decomposition - non square cases

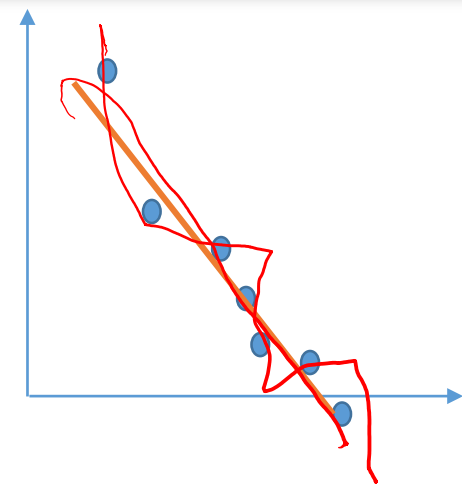
$$\begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \\ 0 & 0 & 0 \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{N \times 1} = \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{M \times 1}$$

We see that the triangular structure allows us to get a result for $\mathbf{x}$ from just the first 3 rows, but then we have "nothing left" with which to satisfy the last equation

It can be shown that: The "solution" we get for $\mathbf{x}$ is the "best fit solution" - now doesn't that sound engineeringly useful - not a perfect solution, but I assure you that the one I present you is the best possible.

A picture? For $N=2$, it is exactly the case of finding a single straight line passing through $M>2$ points.

Think signal processing, design optimisation to meet many criteria - any "best if not perfect" type problem



QR Decomposition - non square cases

Example: Apply the QR decomposition to the problem of solving the underspecified problem $A^{M \times N} x = b$ ($M < N$) and try to give a picture to interpret the outcome

Solution:

$$\begin{pmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{N \times 1} = Q^{M \times M} \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{N \times 1} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{M \times 1}$$

A x Q R x b

As usual, we can use the orthogonality of Q

$$\begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{N \times 1} = Q^T \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{M \times 1} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{M \times 1}$$

b b'

The right hand side is trivial to evaluate and completely known

The problem here is that we don't have that "bottom row with only one non-zero element" to start the triangular solve from! However - I know a trick for this!

QR Decomposition - non square cases

$$\begin{pmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{pmatrix}^{M \times N} \quad \mathbf{Q}^{M \times M} \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & * & * \end{pmatrix}^{M \times N}$$

$\mathbf{A} \qquad \mathbf{Q} \qquad \mathbf{R}$

Trick of the trade: Don't do QR on matrix $\mathbf{A}$, rather on its transpose, $\mathbf{A}^T$

$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \\ * & * & * \end{pmatrix}^{N \times M} = \mathbf{Q}^{N \times N} \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \\ 0 & 0 & 0 \end{pmatrix}^{N \times M}$$

$\mathbf{A}^T \qquad \mathbf{Q} \qquad \mathbf{R}$

Different $\mathbf{Q}$ and $\mathbf{R}$ to above

Now transpose both sides to get

$$\begin{pmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{pmatrix}^{M \times N} = \begin{pmatrix} * & 0 & 0 & 0 \\ * & * & 0 & 0 \\ * & * & * & 0 \end{pmatrix}^{M \times N} \mathbf{Q}^{N \times N}$$

$\mathbf{A} \qquad \mathbf{R}^T \qquad \mathbf{Q}^T$

Now we can proceed again

QR Decomposition - non square cases

$$\begin{pmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{N \times 1} = \begin{pmatrix} * & 0 & 0 & 0 \\ * & * & 0 & 0 \\ * & * & * & 0 \end{pmatrix}^{M \times N} Q^{N \times N} \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{N \times 1} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{M \times 1}$$

A x R^T Q^T x b

Let us define $Qx = x'$ and hence note $x = Q^T x'$

Solve $R^T x' = b$, i.e.

$$\begin{pmatrix} * & 0 & 0 & 0 \\ * & * & 0 & 0 \\ * & * & * & 0 \end{pmatrix}^{M \times N} \begin{pmatrix} * \\ * \\ * \\ * \end{pmatrix}^{N \times 1} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}^{M \times 1}$$

Following the usual lower triangular solve routine, top down, we see that we don't have any equation involving the last element of x' - it is always multiplied by 0.

So the form of the solution is $x' = \begin{pmatrix} \text{calculated} \\ \text{calculated} \\ \text{calculated} \\ \text{Arbitrary} \end{pmatrix}$

The use of the QR algorithm cleanly isolates the arbitrariness in the solution due to the underspecification of the problem

QR Decomposition - non square cases

$x' = \begin{pmatrix} \text{calculated} \\ \text{calculated} \\ \text{calculated} \\ \text{Arbitrary} \end{pmatrix}$ so that the actual solution we seek is $x = Q^T x'$

We might like to think that there is a basic minimal solution $x' = \begin{pmatrix} \text{calculated} \\ \text{calculated} \\ \text{calculated} \\ 0 \end{pmatrix}$

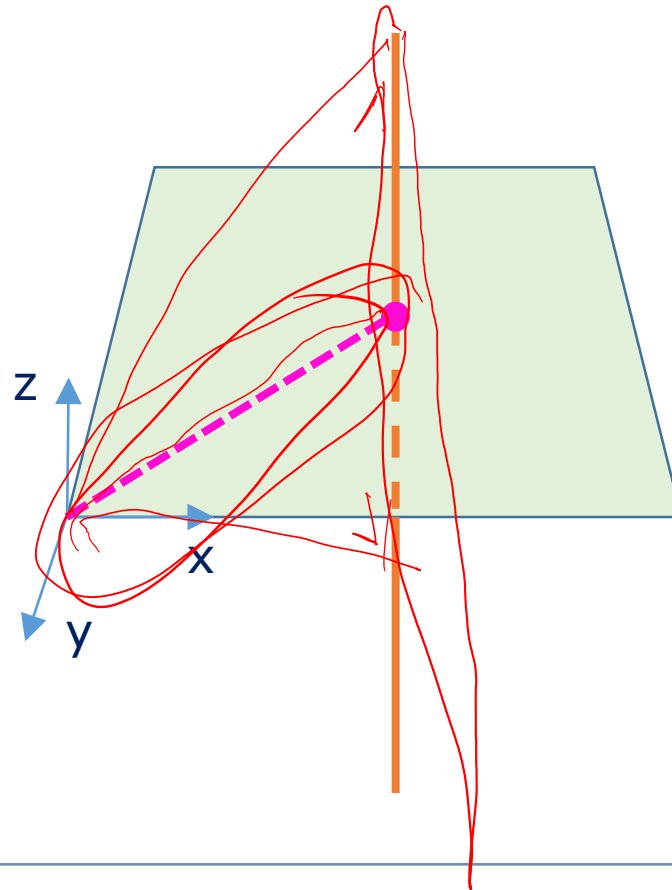
and if we choose to involve the arbitrary term, we get alternative solutions with larger magnitudes of x' and hence x . In fact

It can be shown that: Of all the infinitely possible solutions of an underspecified problem selecting the "arbitrary values" to be zero gives the smallest possible norm solution, i.e. $\|x\|_2^2 = x^T x$ is minimised

Often in some sense $\|x\|_2$ represents a total "cost" - e.g. sum of squares of voltage which is power, so an engineer might well have a sensible meaning for this

A picture ?

QR Decomposition - non square cases



Analogy: Solving in a 3D world, but only given two equations - so anywhere along the vertical line is a legitimate solution to the two equations, but the pink one is that with the smallest magnitude

Can we meaningfully solve non-square matrix problems?

We have shown that QR decomposition applies to non-square problems and now we have an engineering sense of what their "solutions" mean; they are definitely useful

How can we protect ourselves against all the computer rounding errors for large problems?

The impact of numerical rounding error on finite precision CPUs has been a recurrent concern

We have also expressed concern that a square matrix problem may not actually have as many "independent equations" as it seems

QR Decomposition - Rank

A major concern for numerical calculations is "sensitivity"

If we have a matrix problem $Ax = b$ and make a "tiny" perturbation in the elements of A , it is possible for the solution, x , to change "a lot more" than one might expect

This issue is referred to as one of ***sensitivity*** or ***stability***

A typical way of thinking about this is that we have N equations in N unknowns, but some of the equations are "virtually the same in some sense" - and the slightest perturbation makes the problem underspecified.

We saw before, underspecified, permits an "arbitrary amount" of some quantity to be added to the "minimal solution" - it is the wild variations in the amount of this that causes instability

Proposal: The QR algorithm "isolates the arbitrariness" - sounds relevant?

QR Decomposition - Rank

Example: Review the square case QR approach to solving $Ax = b$ and consider where it might "fail"

Solution:

$$\begin{pmatrix} * & * & * \\ * & * & * \\ * & * & * \end{pmatrix} \begin{pmatrix} * \\ * \\ * \end{pmatrix} = Q \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \end{pmatrix} \begin{pmatrix} * \\ * \\ * \end{pmatrix} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}$$

$A \quad x \quad Q \quad R \quad x \quad b$

$$\begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \end{pmatrix} \begin{pmatrix} * \\ * \\ * \end{pmatrix} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}$$

$R \quad x \quad Q^T b$

$$\begin{pmatrix} * & * & * \\ 0 & 0 & * \\ 0 & 0 & * \end{pmatrix} \begin{pmatrix} * \\ * \\ * \end{pmatrix} = \begin{pmatrix} * \\ * \\ * \end{pmatrix}$$

Up to here is numerically safe, but now when we solve the triangular problem, what if one or more diagonal elements were 0? We cannot proceed with certainty

QR Decomposition - Rank

It is easy to show that every 0 we get on the diagonal of the triangular matrix is behaving as if we had removed one of the row equations of the original $\mathbf{Ax} = \mathbf{b}$ problem statement. So although we claimed to have an N by N matrix, $\mathbf{A}$, it behaves like an N-1 by N one if we have one 0 on the diagonal of $\mathbf{R}$.

We will define: A critically important quantity as the "effective" size of the matrix called its **Rank**.

If we have a square matrix, but actually the rows are not independent, then we will get 0s on the diagonal of $\mathbf{R}$. Thus a QR decomposition is a way of answering the question - do we have enough independent equations to permit a unique solution?

In practice, numerical rounding errors perturb these 0s to very small numbers. If we didn't do anything, the triangular solve does division by them - leading to *big* errors. As QR isolates them, we can actively apply an "ignore if below threshold policy" and explicitly exclude the corresponding contributions to the overall solution

QR Decomposition - Rank

Example: Show that the matrix $\begin{pmatrix} 3 & 9 & -1 \\ 4 & 8 & 2 \\ 8 & 16 & 4 \end{pmatrix}$ (slide 53) is *Rank Deficient*

How can we protect ourselves against all the computer rounding errors for large problems?

QR decompositions (and later SVD) can be used to *reveal* the effective number of equations the matrix actually contains - not always the same as the number of rows

We can positively exclude problematic numerical behaviour by examining the QR (or SVD) decomposition; however this is not "cheap"